

★ WEEK 12 · LECTURE

Teaching Machines to Read

NLP, large language models, RAG — and a finance chatbot

Expanded edition · 38 slides · with three hands-on labs

The technology behind ChatGPT, explained from the ground up. We turn words into numbers, meet the giant autocomplete that is an LLM, and fix its biggest flaw with RAG.

Where We Are Going Today

Week 8's neural networks are the engine. This week we point them at language

1

Turning words into numbers (NLP)

Machines only read numbers. Embeddings place words on a map where meaning = location.

2

The giant autocomplete (LLMs)

How models like ChatGPT work — next-word prediction at massive scale — and where they go wrong.

3

The open-book fix (RAG)

Stop the model from making things up by letting it look up your real documents first.

By the end you will measure meaning with an angle, run a softmax by hand, and retrieve like a RAG.

1

PART 1 · WORDS TO NUMBERS

Turning Words into Numbers

NLP and embeddings

Machines Only Understand Numbers

So the very first step is translation

Every model we've built took numbers as input. But language is words, not numbers. So before any AI can 'read', we must turn words into numbers — cleverly, so meaning survives.

Two steps

- Tokenize: chop text into small pieces (tokens).
- Embed: give each piece a list of numbers (a fingerprint).
- Those numbers are called an 'embedding'.

The clever part

- The numbers aren't random.
- Words with similar meaning get similar numbers.
- So meaning becomes distance — and distance we can compute.

One Sentence's Journey Into Numbers

'The Fed raised rates.' — three stops from ink to arithmetic

1 · Tokenize

chop into pieces

'The Fed raised rates.' becomes small chunks: The / Fed / raised / rates / . — words, sub-words, punctuation.

5 tokens

2 · Look up IDs

each token gets a number

Every token has a slot in the vocabulary: The→464, Fed→281, raised→1902... just an address, no meaning yet.

a list of integers

3 · Embed

IDs become fingerprints

Each ID maps to a learned vector of hundreds of numbers — THIS is where meaning lives, as position.

rates $\approx (2, 1, \dots)$ in our toy

Everything downstream — similarity, search, the LLM itself — is arithmetic on stop 3's vectors.

Embeddings — a Map of Meaning

Similar words get similar number-fingerprints, so they land near each other. 'Find similar text' = 'find nearby points'.

Words to Numbers to a Map of Meaning



Measuring How Close Two Meanings Are

Cosine similarity: the angle between two fingerprints

$$\text{similarity} = \cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

Reading it

- a and b are two words' (or sentences') number-fingerprints.
- We measure the ANGLE between them, not the distance.
- Small angle (cosine near 1) = very similar meaning.

Why this is so useful

- 'Find related news' = find high-similarity articles.
- 'Search by meaning, not keywords' becomes possible.
- It's the engine inside search, recommendations, and RAG.

Cosine Similarity by Hand

Four toy sentence-fingerprints in 2-D — the whole geometry of meaning (full version: problem set)

Pair	Dot product $\mathbf{a} \cdot \mathbf{b}$	Norms $ \mathbf{a} \cdot \mathbf{b} $	Cosine	Reading
A (2,1) vs B (4,2)	$8 + 2 = 10$	$\sqrt{5} \cdot \sqrt{20} = 10$	1.00	same direction — paraphrases
A (2,1) vs D (0,3)	$0 + 3 = 3$	$\sqrt{5} \cdot 3 \approx 6.7$	0.45	related, not identical
A (2,1) vs C (-1,2)	$-2 + 2 = 0$	$\sqrt{5} \cdot \sqrt{5} = 5$	0.00	perpendicular — unrelated topics

A = 'Fed raised rates', B = 'central bank hiked', C = 'refund policy', D = 'diversification'. The angle sees what spelling can't.

You Can Do Math With Meaning

The famous example that amazed researchers

Take the fingerprint for 'king', subtract 'man', add 'woman' — and you land almost exactly on 'queen'. Meaning has become arithmetic. That's how powerful good embeddings are.

What it shows

- Embeddings capture real relationships, not just spelling.
- 'Royalty', 'gender', and other ideas live as directions.
- The model learned these from reading, never being told.

In finance

- Embed news headlines, filings, earnings calls.
- Group similar companies or topics automatically.
- Spot when today's news 'sounds like' a past crisis.

2

PART 2 • LLMs

The Giant Autocomplete

Large language models

From Autocomplete to ChatGPT

One simple goal, scaled up beyond imagination

"Train a neural network to do one boring task — guess the next word — on a huge chunk of the internet. Out came something that can write, code, and reason."

The surprise

- Nobody told it grammar, facts, or logic.
- To predict the next word well, it had to learn them.
- Scale unlocked the magic — the Transformer era, from about 2017.

Why it matters to us

- LLMs can read filings, summarize news, draft reports.
- They power the finance chatbot we're building toward.
- But they have a dangerous flaw we must fix (RAG).

The Whole Trick in Four Numbers

A boring objective, an old engine, a new wiring — and one honest zero

1

Training task: guess the next word. That's the entire curriculum

2017

'Attention Is All You Need' — the Transformer wiring arrives

W8

The engine underneath: the same neurons, layers, and backprop

0

Databases consulted while it answers you — it GUESSES from patterns

Keep the zero in mind — it is the seed of hallucination, and the reason RAG exists.

An LLM Is a Giant Autocomplete

It predicts the next word, picks one, and repeats. Week 8's neural network, scaled enormously, wired as a Transformer.

An LLM Is a Giant, Very Well-Read Autocomplete



What an LLM Actually Computes

A probability for every possible next word

LLM: $\Pr(\text{next word} \mid \text{all previous words})$

Reading it

- Given everything written so far, it scores each possible next word.
- It picks a likely one, adds it, and repeats.
- Do this thousands of times and you get a whole essay.

The key realization

- It is NOT looking up answers in a database.
- It is GUESSING the next word from patterns it learned.
- That's why it's fluent... and why it can be confidently wrong. Fluency is not truth.

One Next-Word Choice, by Hand

'Yesterday the market ____' — three candidates, scores $z = (\text{rose } 2, \text{fell } 1, \text{banana } -1)$ (full version: problem set)

Candidate	Score z	e^z (helper)	Probability $e^z / 10.48$
rose	2	7.389	70.5%
fell	1	2.718	25.9%
banana	-1	0.368	3.5%

The softmax turns scores into a betting slip over the whole vocabulary — then the model draws one word and moves on.

Temperature — the Personality Dial

Divide the scores by T before the softmax: same knowledge, different nerve

Temperature	P(rose / fell / banana)	Behavior	When you'd want it
T = 0.5 (cold)	87.9 / 11.9 / 0.2 %	sharp, near-deterministic	facts, numbers, compliance answers
T = 1 (neutral)	70.5 / 25.9 / 3.5 %	the raw learned odds	general chat
T = 2 (hot)	54.7 / 33.1 / 12.2 %	flat, adventurous — banana rises 3×	brainstorming, creative drafts

For a finance chatbot, run cold: a 12% chance of 'banana' is not a feature when the topic is someone's account.

Attention — the Transformer's Trick

For each word, focus on the other words that give it meaning. Attention is why LLMs handle long context so well.

Attention — Focus on the Words That Matter

"I sat on the

bank of the **river**"



"bank" looks at "river"

The problem with words

"Bank" could mean a money bank or a river bank. The word alone is unclear.

Meaning depends on the OTHER words around it.

What attention does

For each word, the model decides which other words to focus on.

Here, "bank" pays attention to "river",
so it knows: the river kind. (Transformer)

The Danger: Hallucination

LLMs can be confidently, fluently wrong

Because an LLM guesses plausible words, it can invent fake facts, fake numbers, even fake citations. It says them with total confidence. In finance, a confident wrong answer can be very costly.

Why it happens

- The model optimizes for plausible, not true.
- Its knowledge is fuzzy and frozen at training time.
- Ask about your private or recent data — it has none.

The fix (Section 3)

- Don't trust its memory.
- Make it look up real documents first.
- Then answer only from what it found. That's RAG.

Three Small Disasters, One Root Cause

Each sounded perfect. Each was invented. Plausible $\neq$ true

The phantom citation

research note

An analyst asks for supporting literature. The model supplies a beautifully formatted paper — authors, journal, year. It does not exist.

caught only when compliance clicked the link

The invented EPS

earnings question

'What was Q3 EPS?' The model answers '\$2.41' fluently. The filing says \$1.87. The model never opened the filing — it guessed a plausible number.

a confident number is not a sourced number

The imaginary policy

customer chat

'Can I withdraw early?' The bot describes a generous policy that was never written — trained on OTHER banks' pages.

fluent, kind, and wrong = a complaint letter

None of these are bugs — they are the autocomplete doing its job. The cure is grounding, not scolding.

3

PART 3 · RAG

The Open-Book Fix

Retrieval-Augmented Generation (RAG)

RAG — Look It Up, Then Answer

Retrieve the relevant document, hand it to the LLM, then generate. An open-book exam beats a memory test.

RAG — Give the Model an Open Book Before It Answers



Why this fixes hallucination

A plain LLM answers from fuzzy memory and may confidently invent facts — deadly in finance.
RAG turns it into an OPEN-BOOK exam: look up the real document first, then answer and cite it.

Bonus: it can use your private, up-to-date files — which the model was never trained on.

How RAG Works, Step by Step

Embeddings from Section 1 do the heavy lifting

1

Embed your documents

Turn every page of your trusted files into number-fingerprints (embeddings) and store them.

2

Find the closest pages

Embed the user's question, then use cosine similarity to grab the most relevant pages.

3

Answer from those pages only

Hand them to the LLM with 'answer using only this'. Grounded in real facts — and it can cite them.

RAG = your Section-1 embeddings + your Section-2 LLM, working as a team.

One Retrieval, by Hand

Question: 'How long do I have to return something?' — keyword-count vectors over (rates, inflation, refund, return, earnings)

Document	Vector	$\cos(\mathbf{q}, \mathbf{d})$ with $\mathbf{q} = (0,0,0,1,0)$	Verdict
d_1 · Fed raised rates to fight inflation	(1,1,0,0,0)	$0 / (1 \cdot \sqrt{2}) = 0$	irrelevant
d_2 · Refund policy: returns within 30 days	(0,0,1,1,0)	$1 / (1 \cdot \sqrt{2}) = 0.707$	RETRIEVED ✓
d_3 · Quarterly earnings beat expectations	(0,0,0,0,1)	0	irrelevant

Now hand d_2 to the LLM with 'answer ONLY from this' — the guessing machine becomes a reading machine (full version: problem set).

Why Finance Needs RAG

Three reasons it's the standard for serious tools

Accuracy

grounded answers

Answers come from real documents — far fewer made-up facts, and it can show its source: auditable.

cite the filing, not a vibe

Freshness

no retraining

Use today's filings and prices. Update the documents, not the model.

new PDF in, new answers out

Privacy

your files stay yours

Use your own private files the base model never saw. Sensitive data stays under your control.

the corpus is the moat

This is how real finance chatbots answer questions about policies, filings, and accounts safely.

When RAG Still Fails — and the Fix Each Time

Grounding is a system, and every part can leak

Failure	What it looks like	The fix
Wrong page retrieved	fluent answer, from the wrong document	better chunks; rerank top-k; test retrieval alone
Answer isn't in the corpus	the model 'helpfully' falls back to guessing	instruct: 'if not found, say NOT FOUND'
Stale index	yesterday's policy, quoted confidently	re-embed on every document update
Chunk cut mid-thought	half a rule retrieved, half invented	overlap chunks; keep sections whole
Prompt ignored	answers beyond the pasted context	lower temperature; demand citations per claim

The auditor's question survives from Week 10: where EXACTLY did this number come from?

4

PART 4 · HANDS ON

Your Turn — Hands On

Build the pieces of a finance chatbot

The Finance Chatbot, Assembled

Every piece from today, working together

1

User asks a question

'What were our Q3 earnings?' or 'Summarize this 10-K.' Plain language, no code.

2

RAG retrieves the real answer

Embed the question, search the trusted filings, pull the exact relevant passages.

3

LLM writes a grounded reply, with a citation

It answers from those passages only — fluent, helpful, and traceable back to the source.

Fluent LLM + honest retrieval = a chatbot you can actually trust with money questions.

The Finance NLP Menu

Same three ingredients — embeddings, LLM, retrieval — plated four ways

Dish	Ingredients	What it does on a desk
Sentiment scoring	embeddings + a classifier (W5)	score headlines bullish/bearish as a signal input
Summarization	LLM	1,000 pages of filings → one page per company
Similarity search	embeddings + cosine	'find past episodes that SOUND like today'
Grounded Q&A chat	embeddings + LLM + RAG	the trustworthy chatbot — today's capstone

Every dish is arithmetic on Section 1's vectors — the map of meaning is the kitchen.

Which Piece of the Stack? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which piece / diagnosis?
'Find filings that resemble this paragraph'	Embeddings + cosine — nearest points on the map
The bot cites a paper that doesn't exist	Hallucination — plausible $\neq$ true; ground it
Answers must use TODAY's rate sheet	RAG freshness — update the documents, not the model
Compliance answers vary run to run	Temperature too high — run the softmax cold
Bot answers even when the corpus is silent	Add the 'say NOT FOUND' instruction — RAG hygiene

If you got 4 of 5, the chatbot architect's instinct of Week 12 is yours.

Getting This Week's Data

Text this week — a tiny finance corpus built in the notebook

A built-in text set (no download)

- This week is about language, not price tables.
- A handful of finance sentences, defined right in the lab.
- Real systems use neural embeddings; we use TF-IDF to learn.

What we'll do

- Lab 1: embed sentences and find the most similar pair.
- Lab 2: a tiny RAG — retrieve the best-matching document.
- All with scikit-learn's TF-IDF + cosine similarity.

[CODE] colab_setup.py · tiny finance text (built inline)

```
docs = ['The Federal Reserve raised interest rates to fight inflation.',  
        'The central bank hiked rates again to slow inflation.',  
        'Our refund policy allows returns within 30 days of purchase.',  
        'You have 30 days to return an item for a refund.',  
        'Diversifying a portfolio lowers risk without lowering return.',  
        'Quarterly earnings beat analyst expectations this period.']
```

Hands-On Lab 1 — Find Similar Sentences

Turn text into numbers, then measure closeness

[CODE] colab_lab1.py · tiny finance text (built inline)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

vecs = TfidfVectorizer().fit_transform(docs) # text -> nums
sim = cosine_similarity(vecs)               # how alike?

np.fill_diagonal(sim, 0)                    # ignore self-matches
i, j = np.unravel_index(sim.argmax(), sim.shape)
print('most similar pair:')
print(' -', docs[i])
print(' -', docs[j])
```

Your task (step by step)

- Turn the docs into number vectors (TF-IDF).
- Compute cosine similarity between them.
- Find the most similar pair.
- Check: do they really share meaning?
- Q: does the math match your intuition?

Hands-On Lab 2 — A Tiny RAG Retriever

Answer a question by finding the right document

[CODE] colab_lab2.py · tiny finance text (built inline)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vec = TfidfVectorizer().fit(docs)
doc_vecs = vec.transform(docs)

question = 'How long do I have to return something?'
q_vec = vec.transform([question])
best = cosine_similarity(q_vec, doc_vecs).argmax()

print('Question:', question)
print('Retrieved:', docs[best]) # feed THIS to an LLM
```

Your task (step by step)

- Embed the docs AND a user question.
- Find the doc closest to the question.
- Return it as the grounded 'answer'.
- Try a few questions and watch it retrieve.
- Q: would a real LLM now answer from this doc?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 and Lab 2 code above, one plain-English sentence per line.
- 2) Grow the corpus to 12 finance sentences (invent sensible ones); rebuild the retriever; run 5 test questions and print a retrieval table with scores.
- 3) Add the honesty rule: if the best cosine is below 0.15, print 'NOT FOUND in corpus' instead of a doc. Show one question that triggers it.
- 4) Build the full RAG prompt string for one question: retrieved passage + 'Answer ONLY from the text above; cite it; if absent, say NOT FOUND.'
- 5) In 3 sentences: why the threshold in step 3 is the anti-hallucination valve, in this week's language.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 What is an embedding?

- A A type of chart
- B A list of numbers capturing a word's meaning ✓**
- C A trading strategy
- D A neural network layer

Answer: An embedding is a number-fingerprint for text, placed so similar meanings sit close together.

Q2 How does RAG reduce hallucination?

- A It makes the LLM bigger
- B It retrieves real documents and answers from them ✓**
- C It shuffles the text
- D It removes the LLM

Answer: RAG looks up trusted documents first, so the model answers from real facts instead of fuzzy memory.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 Your compliance bot gives different answers each run. First knob to turn:

- A** Add more documents
- B** Lower the temperature — sharpen the softmax ✓
- C** Raise the temperature
- D** Use longer prompts

Answer: Cold T concentrates probability on the top choice: 70.5% became 87.9% at $T = 0.5$ in our hand-walk.

Q4 Best cosine between the question and ALL corpus docs is 0.08. The bot should:

- A** Answer from the closest doc anyway
- B** Say 'NOT FOUND in corpus' ✓
- C** Raise the temperature
- D** Invent a plausible answer

Answer: Nothing relevant was retrieved — answering anyway is hallucination with extra steps. The threshold is the valve.

Week 12 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
Pipeline	tokenize → IDs → embeddings; meaning lives at stop 3, as position.
Cosine	$a \cdot b / a b - 1$ = paraphrase, 0 = unrelated; the engine of search & RAG.
LLM	a giant autocomplete: softmax over the vocabulary, one word at a time.
Temperature	divide scores by T: cold = sharp and factual, hot = flat and creative.
Hallucination	plausible $\neq$ true; the model consults 0 databases while answering.
RAG	embed → retrieve by cosine → 'answer ONLY from this' → cite; add a NOT-FOUND valve.

Next stop: agents that learn to ACT — reinforcement learning and smart execution.

Three Ideas, One Picture Each

The theme: fluency is not truth. Ground the model in real data, and it becomes trustworthy

1

Embeddings = a map of meaning

Words become number-fingerprints where similar meanings sit close. Cosine similarity measures 'how alike?'.


2

An LLM is a giant autocomplete

It predicts the next word at huge scale (Week 8 nets + Transformers). Fluent, but can confidently make things up.


3

RAG makes it open-book

Retrieve your real documents, then answer from them. Accurate, fresh, private — the recipe for a finance chatbot.


Keep the pictures: a map of meaning, a betting slip over words — and an open-book exam.

Week 12 — Three Things to Remember

Before next class: finish Labs 1–3 and try your own question in the tiny RAG retriever

1

Text becomes numbers (embeddings)

Tokenize, then embed. Meaning becomes location; cosine similarity finds related text. Powers search and RAG.

2

LLMs predict the next word

A giant autocomplete built on Week 8 neural nets. Powerful and fluent, but prone to confident hallucination.

3

RAG grounds the answer

Retrieve trusted documents, then generate from them. Accurate, up-to-date, private — the heart of a finance chatbot.

Next week — Reinforcement Learning: agents that learn to act, and smart trade execution.